Project Plan

Pralax - an Eye-Tracked Parallax Viewer

Group 3 - Project 2.1
Maksymilian Zalewski (6444754), Justin Abel (6434415)
Rotem Aharoni (6442833), Ginguta Tudor Dan (6442826)
Federico D'Agata (6445526), Sotiris Andronikos Mitsopoulos (6344396) [1.5mm] Tutor: Vasileios Tzatz

October 9, 2026

1 Introduction

Pralax is a desktop application that brings 2D artwork and 3D scenes to life. The way it works is by taking the content on screen and having it respond to the viewer's eye position, creating a parallax effect - as if looking through a window into the scene. The application is implemented as a Tauri desktop app (Rust shell, SvelteKit + TypeScript frontend). It currently offers one mode, parallaxed 2D artworks rendered as depth layers; if time allows, we will add two further modes: (*i*) 3D scenes created in Blender, and (*ii*) user-uploaded images projected into a parallax view. The eye tracking is performed with a regular webcam, so no dedicated hardware is required.

This document presents the results of the first project phase: the envisioned software architecture (Section 2), a comparison of the software components we tested and the approaches we considered, including the performed tests and the selection criteria (Section 3), the task planning for the second and third phases with responsibilities (Section 4), a Gantt chart of the main tasks (Section 5), and a risk analysis (Section 6).

2 Software Architecture

Figure 1 shows the target architecture as an annotated block diagram. Green boxes mark components that already exist as phase-1 prototypes, blue boxes are built in phases 2 and 3, and grey boxes are hardware.

Data flow. The webcam frames are used by a Python sidecar process running MediaPipe Face Mesh. The module estimates the viewer's face/eye position, applies the calibration and exponential smoothing, and publishes a normalized eye position (x, y) as JSON events to the front-end. The Rust Tauri shell manages the sidecar lifecycle and exposes a *localhost-only* IPC bridge, which keeps the tracking data inside the machine (no network exposure); the messages are small (two coordinates and a timestamp), so JSON encoding adds latency, but it is so small that it is negligible. The SvelteKit UI receives the events and passes the position, together with the user settings (intensity, depth, smoothing - the parameters already prototyped in the phase-1 overlay), to the parallax engine. The engine renders either a 3D scene (glTF assets exported from Blender, rendered with Three.js and a camera offset proportional to the eye position) or a 2.5D artwork (HTML/CSS depth layers that move at different rates). Scene components are loaded lazily so that modes can be added without restarting the app.

Status after phase 1. The eye-tracking controller API (Tudor), the Tauri window system with lazy loading and the settings overlay (Federico), the first Blender scene (Justin, Rotem) and the face-to-parallax mapping prototype (Sotiris) already exist; we also researched the build system and CI/CD set-up. The IPC bridge, the mode selection UI, the parallax engine, the CI pipeline and the asset pipeline are built in phase 2 (Section 4).

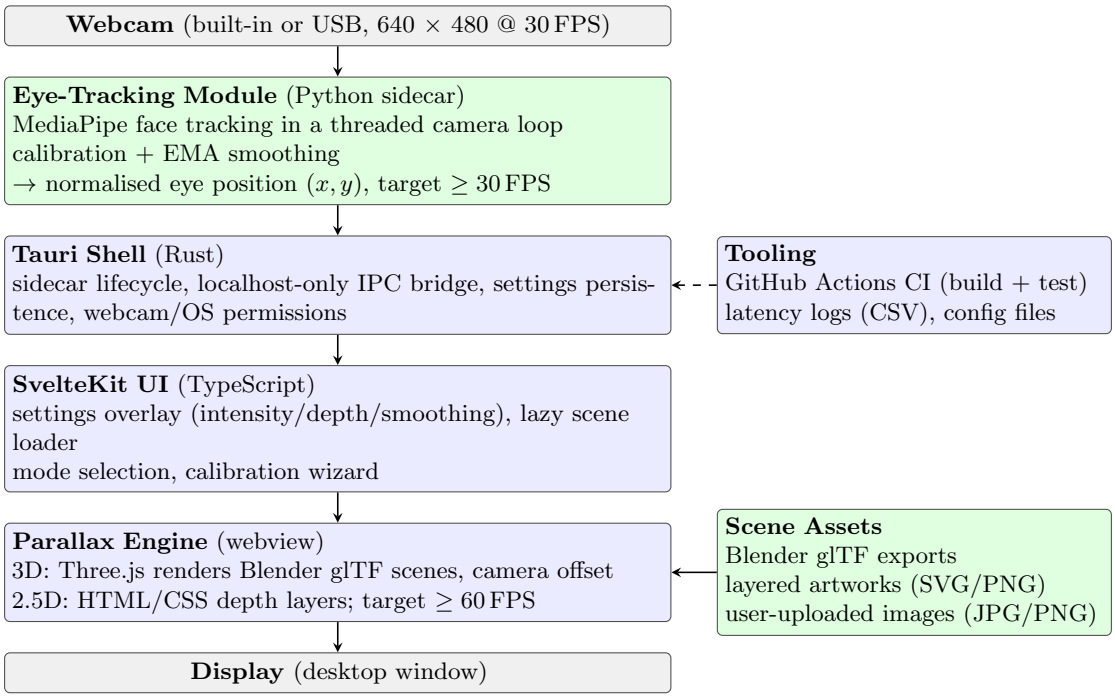


Figure 1: Annotated block diagram of the envisioned software architecture. Green: phase-1 prototype exists; blue: built in phases 2–3; grey: hardware. Solid arrows show the run-time data flow, the dashed arrow shows build and test tooling.

3 Software Component Tests and Comparison

3.1 Evaluation criteria and test methodology

All candidate components were evaluated against the same five criteria:

- **C1 - Responsiveness:** frame rate and end-to-end latency (camera to screen), because a laggy parallax effect is immediately perceived as broken.
- **C2 - Accuracy and robustness:** correct detection under realistic conditions (glasses, varying lighting, head movement), which determines how many users can actually use the app.
- **C3 - Integration effort:** how easily the component connects to our chosen stack (Tauri webview, SvelteKit/TypeScript, Python sidecar), including data formats and licensing friction.
- **C4 - Resource usage:** CPU/GPU load, since tracking and scene rendering must run concurrently on consumer hardware.
- **C5 - Maintainability:** documentation, community activity, permissive licensing and the team's familiarity with the technology.

Test setup. Measurements were taken on the group's development laptops with a 640×480 webcam at 30 FPS. Each run captured 60 s of video; the camera-loop frame rate and the per-frame detection latency were measured with timestamps. Robustness was assessed qualitatively on the team members (with and without glasses, different rooms and lighting).

3.2 Eye tracking

Two eye-tracking implementations were built and tested in phase 1.

1. **MediaPipe Face Mesh (Python).** MediaPipe is Google's open-source framework for on-device perception. It ships pre-trained machine learning models (`.tflite`) and exposes a Python API; its face pipeline first locates the face with the BlazeFace short-range detector and then fits a 468-point face mesh that includes the eye landmarks. We built a threaded camera

loop using the `LIVE_STREAM` running mode with an asynchronous result callback, so detection runs concurrently with the GUI.

2. **OpenCV (C++).** OpenCV is the standard open-source computer vision library; we built a C++ demo using its pre-trained Haar cascade-based face detector. The fact that it was written in C++, led us to expect that it would offer a speed advantage over MediaPipe Face Mesh.

Table 1: Speed comparison of the tested face-tracking components.

Metric	**MediaPipe (Python)**	**OpenCV (C++)**
Camera loop rate	29.1 FPS	29.8 FPS
Detection rate	29.1 FPS	29.8 FPS
Average detection latency	2.3 ms	2.1 ms

Results. As Table 1 shows, both libraries performed nearly identically: the 30 FPS webcam is the bottleneck, and both detectors saturate it. The OpenCV demo was marginally faster (+0.7 FPS, −0.2 ms), but the qualitative robustness test revealed a decisive difference: when the user wore glasses, the OpenCV demo detected the face and eyes only intermittently - from time to time it found something, while without glasses the detection was more or less instant. MediaPipe kept detecting reliably in both cases. Since the speed advantage was negligible, we saw no benefit in spending time calibrating the C++ library to reach MediaPipe's robustness. We therefore **select MediaPipe Face Mesh (Python)** for the eye-tracking module. It additionally provides 468 facial landmarks, including eye landmarks, which allow us to refine the eye-position estimate in later phases, and it integrates directly with the phase-1 controller API (C3, C5).
Considered but not selected. Dedicated hardware eye trackers (e.g. Tobii) were rejected because they add cost and hardware dependencies, contradicting the requirement to work with a regular webcam (C3, C4). Learned gaze-estimation models (L2CS-Net, GazeML) were rejected as unnecessarily heavy: for a screen-relative parallax effect, the face/eye position suffices, and the extra latency of a deep gaze model does not buy accuracy we can use (C1, C4). Dlib's HOG-based detector was considered but not tested further after MediaPipe's results: it is slower on CPU and less robust than MediaPipe (C1, C2).

3.3 3D scene visualisation

- **Approach A - Blender-authored scenes rendered with Three.js (WebGL) in the webview.** Scenes are authored in Blender (free, large community, familiar to the artists in the team) and exported as glTF, then rendered with Three.js. WebGL runs inside the Tauri webview on all three desktop platforms, and the perspective camera gives full control over the parallax offset.
- **Approach B - Babylon.js.** A full 3D game engine in TypeScript with a richer feature set (PBR, physics, editor). Its added capabilities are largely unnecessary for static scenes, and it adds bundle size and learning cost (C4, C5).
- **Approach C - CSS 3D transforms.** Pure CSS perspective is lightweight, but it cannot import scene formats, has no lighting model and becomes unwieldy for organic scenes (C1, C3).
- **Approach D - 2.5D layered artworks (HTML/CSS).** Artwork is split into depth layers that move at different rates; no GPU work is needed at all. We prototyped this style in phase 1 with placeholder SVG scenes (Mona Lisa, Starry Night) and it matches the intensity/depth/smoothing settings overlay.

Tests performed. In phase 1 we validated (i) the Tauri + SvelteKit window system with lazy scene loading and a controls overlay (Federico), and (ii) the asset pipeline by authoring a first 3D scene in Blender (Justin, Rotem) - which also exposed the export issues (camera placement, colours)

that we address in phase 2. We additionally researched webview-based rendering with HTML/CSS instead of a GPU-rendered library and confirmed both are feasible inside Tauri.
Decision. We **select Approach A** (Blender + Three.js) for the 3D scenes, and **keep Approach D** for the artwork and user-uploaded-image modes, since it needs no GPU and reuses the tested settings overlay. Babylon.js (B) is overkill and CSS 3D (C) is too limited. This hybrid covers all three modes with minimal risk.

3.4 Parallax effect

- **Approach A - Perspective-camera offset.** The eye position translates the 3D camera while the scene stays fixed, producing true parallax with correct occlusion. Naturally compatible with the Three.js renderer.
- **Approach B - Layered displacement (2.5D).** Each depth layer moves proportionally to the eye position divided by its depth. Very cheap, works for artwork and photos, and its tuning knobs (intensity, depth, smoothing) already exist in our overlay.
- **Approach C - Head-coupled off-axis projection.** The frustum is recomputed from the physical screen geometry and the eye's position in front of it (Johnny-Lee/TrackIR style). The most physically correct option, but it requires precise head-position tracking and per-setup calibration; kept as a phase-3 stretch goal if time allows.
- **Approach D - Shader-based parallax (parallax occlusion mapping).** Fakes depth inside a texture; limited to surface detail and unsuitable for real scenes. Rejected.

Tests performed. We built a face-to-parallax mapping prototype in phase 1: the face-tracking output moves the parallax effect relative to a reference point, so that looking at a different screen area changes the view. It works end to end but still needs performance work (the mapping currently runs in the same loop as detection). The intensity/depth/smoothing overlay (Federico) provides the parameter interface for approach B and was tested interactively in the GUI.
Decision. We **select a hybrid of A and B**: camera offset for the 3D scenes and layered displacement for artworks and uploaded images, with EMA smoothing to suppress tracking jitter. Both are cheap (C4), instantly responsive (C1), and built on components already tested in phase 1.

3.5 Application shell

We compared three ways to package the application. **Electron** bundles its own Chromium and Node.js runtime, which makes installers large and raises memory use (C4). A **native desktop toolkit** (e.g. Qt/C++) would give full control but cannot reuse web rendering libraries such as Three.js and increases the implementation effort (C3). **Tauri** uses the operating system's webview with a small Rust core, supports bundling the Python tracker as a sidecar process, and was validated in phase 1 with the window system, lazy loading and settings overlay. We therefore **select Tauri with a SvelteKit + TypeScript frontend** (C3, C4, C5).

3.6 Selection summary

Table 2: Summary of the component selection per domain.

Domain	Considered	Selected	Deciding criteria
Eye tracking	OpenCV (C++), Tobii hardware, gaze models (L2CS/GazeML), Dlib	MediaPipe Face Mesh (Python)	C2 (glasses), C1 parity, C3/C5 (Python API, landmarks)
3D scene visualisation	Three.js (Blender glTF), Babylon.js, CSS 3D, 2.5D HTML/CSS layers	Three.js for 3D scenes + layered 2.5D for artworks	C3/C4/C5; both paths tested in phase 1
Parallax effect	Camera offset, layer displacement, head-coupled frustum, POM shaders	Hybrid: camera offset + layer displacement with EMA smoothing	C1 latency, C4 cost, reuses tested overlay
Application shell	Electron, native desktop, Tauri	Tauri + SvelteKit + TypeScript	C4/C5, tested window system

4 Planned Tasks for Phases 2 and 3

All tasks below are SMART: **Specific** (each names a concrete deliverable), **Measurable** (each has a numeric acceptance criterion), **Achievable** (assigned to members who already built the corresponding phase-1 component), **Realistic** (windows are sized from the recorded phase-1 effort and the agreed split: Federico 20%, Tudor 20%, Sotiris 20%, Justin 15%, Rotem 15%, Max 10% of coding; Max also leads project management and the user study), and **Timely** (fixed windows, reviewed at the weekly project meeting, feeding milestones Oct 11, Nov 28, Jan 30). Tudor co-manages the project. The T2.13 harness logs all measurements so that every experiment is reproducible from a config file.
Milestone M1 (Oct 11): this project plan and the 8-minute demo video (mp4, $\leq$300 MB) of the tested components, submitted to the examiners.

Table 3: Phase 2 - Core implementation (Oct 12 - Nov 28, 2026).

ID	Task	Owner(s)	Window	Acceptance criterion
T2.1	Python ↔ Tauri bridge for tracking data	Federico, Tudor	Oct 12-25	≥30 Hz eye-position events, ≤5 ms latency, localhost-only
T2.2	Render parallax with the Blender scenes in Tauri	Federico	Oct 12-Nov 8	Scenes render at ≥60 FPS; camera offset follows eye position
T2.3	Fix 3D model issues: camera placement, colours, broken parts	Justin, Rotem	Oct 12-25	Asset checklist passed for every scene
T2.4	UI for mode selection (HTML/TypeScript)	Rotem	Oct 12-Nov 1	All modes switchable without reload; settings apply live
T2.5	Tracking performance optimisation	Sotiris, Tudor	Oct 19-Nov 8	Sustained ≥30 FPS; end-to-end latency ≤50 ms
T2.6	Calibration + sensitivity settings	Justin, Sotiris	Oct 26-Nov 15	Calibration ≤1 min; profile persists; sliders affect rendering
T2.7	Extra scenes: artwork parallax + more 3D scenes	Justin, Rotem	Oct 26-Nov 22	≥3 artwork scenes and ≥2 3D scenes of different complexity
T2.8	User-uploaded image mode	Federico, Rotem	Nov 2-22	JPG/PNG import with depth layering, processing ≤2 s
T2.9	CI pipeline (build + test on commit)	Sotiris	Oct 12-25	GitHub Actions builds and tests every commit
T2.10	Eye-tracking accuracy improvements	Tudor, Max	Nov 2-22	Robust with and without glasses; jitter ≤1% of screen width
T2.11	Phase-2 live demo preparation	Max, Tudor	Nov 16-28	20-min demo rehearsed twice; examiners can try the app themselves
T2.12	Peer-feedback session + feedback form	Rotem	Nov 9-22	Form ready; feedback from 2 groups collected and filed as GitHub issues
T2.13	Performance measurement harness	Sotiris	Oct 26-Nov 15	Per-stage timestamps (capture, detection, IPC, render) logged to CSV; runs driven by config files
T2.14	User-study preparation	Justin, Max	Nov 9-28	Protocol, consent form and questionnaire done; 2 pilot runs; participants scheduled

Milestone M2 (Nov 28): end-to-end working prototype - eye movement visibly drives the parallax effect in all three modes.

Table 4: Phase 3 - Testing, polish and delivery (Nov 30, 2026 - Jan 30, 2027).

ID	Task	Owner(s)	Window	Acceptance criterion
T3.1	User study: run sessions and analyse results	Justin, Max	Nov 30-Dec 19	≥10 participants; parallax on/off and scenes of different complexity compared; statistical analysis with plots
T3.2	Performance experiments + optimisation	Sotiris, Tudor	Nov 30-Dec 19	Scene complexity increased until real time breaks; latency histograms and per-component breakdown; ≥30 FPS end to end
T3.3	UI/UX polish	Federico, Rotem	Nov 30-Dec 19	All high-priority examiner, peer and user-study feedback resolved
T3.4	Browser demo for examiners	Sotiris	Dec 7-19	Public URL live with browser-based tracking or mouse fallback; offline build available
-	Christmas break	all	Dec 20-Jan 3	-
T3.5	Bug fixing + code freeze	Federico, Sotiris	Jan 4-17	Zero known critical bugs; release tagged on GitHub
T3.6	Final report (≤10 pages)	Max, Justin	Jan 4-24	Draft reviewed by all members by Jan 17; submitted on time
T3.7	Final presentation + demo prep	Rotem, Tudor	Jan 11-30	5-min talk and live demo rehearsed twice; backup plan ready

Milestone M3 (Jan 30): final report and product examination.

5 Gantt Chart

Figure 2 shows the timeline of the main tasks; task IDs refer to Tables 3 and 4. Weeks start on Mondays, and phase 1 (grey, completed) is included for context.

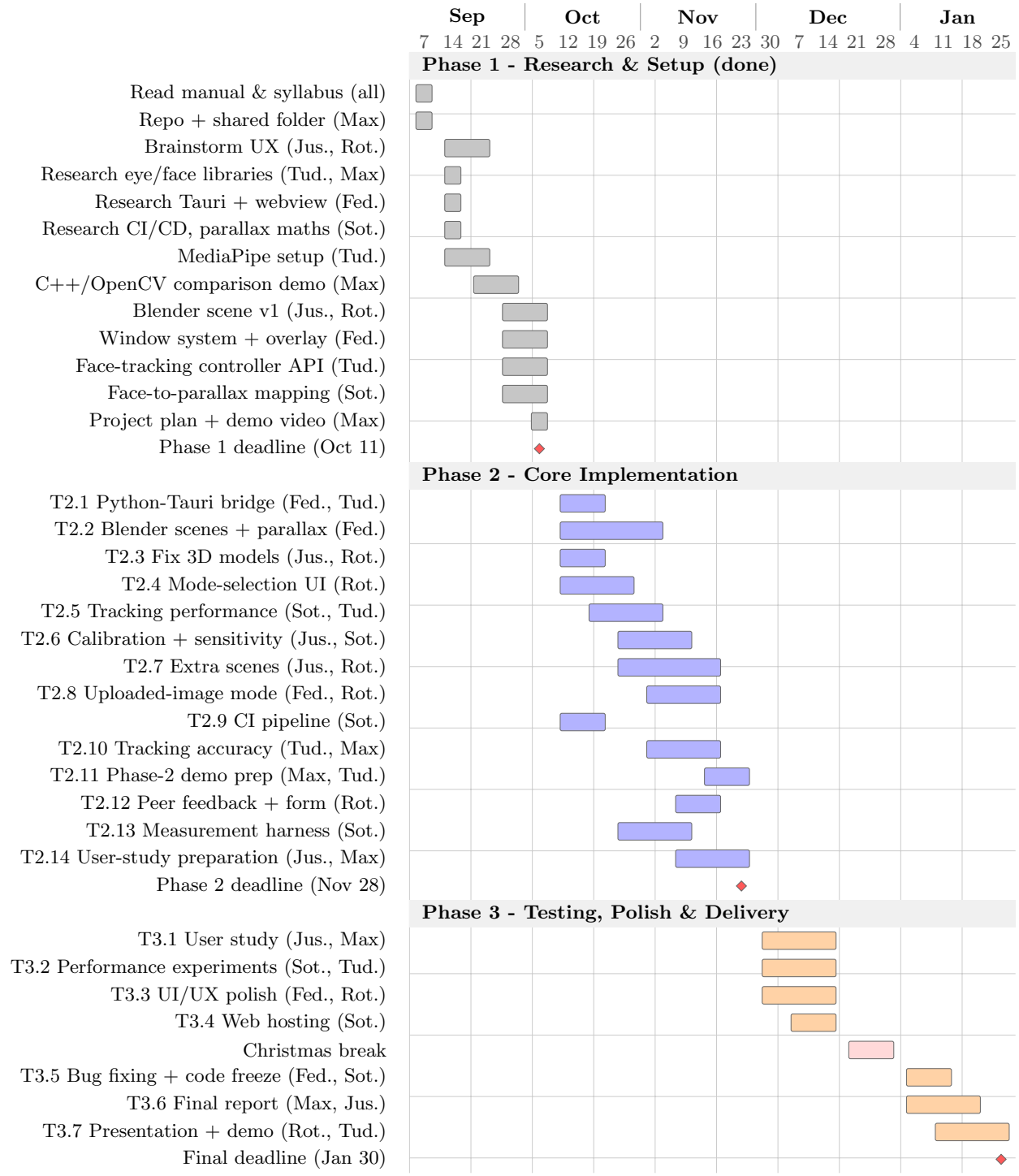


Figure 2: Gantt chart of the main tasks. Grey: phase 1 (completed), blue: phase 2, orange: phase 3, red: break, diamonds: milestones. The number above each column is the date of the Monday that starts the week.

6 Risk Analysis

Table 5 lists the ten major risks of the project, their impact and likelihood (H/M/L), and the mitigation plans. The risks were identified at a group meeting, are reviewed at every weekly project meeting, and are addressed by the tasks referenced in the last column.

Table 5: Risk analysis for the Pralax project.

#	Risk	Impact	Likelihood	Mitigation
R1	Eye tracking fails in poor lighting or with glasses	High: the core feature is unusable for affected users	M	Calibration wizard; lighting guidance in the UI; accuracy task T2.10; head-position fallback (needs less image quality than the eyes); MediaPipe selected for robustness
R2	Latency spikes and jitter from camera to screen	High: the parallax feels laggy and can cause discomfort	M	Latency budget $\leq$50 ms enforced in T2.5 and T3.2; EMA smoothing; threaded pipeline; T2.13 harness detects regressions early
R3	Tauri sidecar IPC bottleneck or insecurity	Medium: integration delay or an exposed local service	M	The bridge is the first phase-2 task (T2.1, early spike); localhost-only bindings; JSON schema validation; CI tests (T2.9)
R4	Blender export issues (materials, cameras, colours)	Medium: scenes render incorrectly, weak visual quality	H	Already observed in phase 1; export checklist and dedicated fix task T2.3; glTF validation; early render test T2.2
R5	Scope creep in the 3D scenes and extra modes	High: missed milestones	M	MoSCoW backlog frozen at phase-2 start; PM reviews scope weekly; stretch items only after M2
R6	Team member illness or absence	Medium: task delays and deadline pressure	M	Paired task ownership; buffer weeks before each milestone; Max (PM) reallocates at the weekly meetings; shared repository and docs
R7	Examiner hardware limitations (weak GPU, no webcam)	High: the demo cannot be shown or runs poorly	M	HTML/CSS layered fallback needs no GPU; quality settings; benchmark on a minimum-spec machine (T3.2); mouse-driven and no-camera fallback modes
R8	Webcam access and permissions differ across OSes	Medium: the app fails on some systems	M	Cross-platform test early in phase 2; permission-request UX; no-camera fallback demo mode
R9	User-study recruitment during the holiday period	Low: fewer or weaker feedback results	M	Sessions scheduled before the break (Nov 30–Dec 19); remote testing via the hosted demo (T3.4); backup participant list
R10	Demo or hosting failure at the final assessment	High: poor grade	L	Host the browser demo early (T3.4); offline desktop build as fallback; rehearsed demo with backup plan (T3.7); screen recording as a last resort